

Start coding or [generate](#) with AI.

UNIT II: Object Oriented Programming Concepts: Class, object, constructor, destructor and inheritance; Modules & Packages, File Input and Output, Catching exceptions to deal with bad data, Multithreading, Database Connectivity.

[+ Code](#) [+ Text](#)

What is a Class?

Definition:

A class is a **blueprint/template** used to create objects.

It defines:

- Variables (data)
- Functions (behavior)

Real-Life Example

Think of a Student Form

- Name
- Marks
- Roll No

This form = Class

Filled form = Object

What is an Object?

Object is a **real instance of a class**

Example:

- Student("Rahul", 85) → object
- Student("Priya", 90) → object

Structure

Class → Blueprint
Object → Real Data

```
class Student:

    # Constructor
    def __init__(self, name, marks):
        self.name = name
        self.marks = marks

    # Method
    def display(self):
        print("Name:", self.name)
        print("Marks:", self.marks)

# -----
# STEP 2: Create Objects
# -----
s1 = Student("Rahul", 85)
s2 = Student("Priya", 90)

# -----
# STEP 3: Use Object
# -----
s1.display()
```

```
s1.display()  
s2.display()
```

What is a Constructor?

📌 Definition (Simple):

A constructor is a **special method in a class** that is **automatically called when an object is created**

Why Do We Use Constructor?

📌 To initialize (assign values to) object variables

➡ Without constructor → we must assign values manually

➡ With constructor → values are set automatically

Syntax of Constructor

Python

```
class ClassName:  
    def __init__(self, parameters):  
        self.variable = value
```



Run

Code	Meaning
init	Constructor function
self	Current object
name, marks	Input values
self.name = name	Store name
self.marks = marks	Store marks

Use of Constructor

Use	Explanation
Initialize data	Set values automatically
Reduce code	No need to assign separately
Clean structure	Organized code
Object creation	Mandatory for data setup

Applications of Constructor

📌 Used in:

- Student Management System
- Banking System (account creation)
- ML Models (initialize weights)
- Web Apps (user data)

Practical Application Example

Python

```
class Bank:  
  
    def __init__(self, name, balance):  
        self.name = name  
        self.balance = balance  
  
    def show(self):  
        print(self.name, "has", self.balance)  
  
b1 = Bank("Rahul", 5000)  
b1.show()
```

📌 Constructor sets account details automatically

Without Constructor (Compare)

```
Python
class Student:
    pass

s1 = Student()
s1.name = "Rahul"
s1.marks = 85
```

Destructor in Python

What is a Destructor?

Definition (Simple):

A destructor is a special method that is automatically called when an object is destroyed (deleted from memory)

Syntax of Destructor

```
Python
class ClassName:
    def __del__(self):
        # cleanup code
```

Example Syntax

```
Python
def __del__(self):
    print("Object destroyed")
```

When Destructor is Called?

Destructor runs when:

- Object is deleted using `del`
- Program ends
- Object goes out of scope

```
class Student:

    def __init__(self, name):
        self.name = name
        print("Constructor called")

    def __del__(self):
        print("Destructor called for", self.name)

# Create object
s1 = Student("Rahul")

# Delete object
del s1
```

Constructor called
Destructor called for Rahul

Explanation

Step	What Happens
Object created	Constructor runs
Object deleted	Destructor runs

Use of Destructor

📁 Used to:

- Free memory
- Close files
- Close database connections
- Release resources

Applications of Destructor

📁 Real-life uses:

- File handling (close file automatically)
- Database connection closing
- Network connection cleanup
- Resource management



[4]
✓ 0s

```
class FileHandler:

    def __init__(self, filename):
        self.file = open(filename, "w")
        print("File opened")

    def write_data(self):
        self.file.write("Hello Students")

    def __del__(self):
        self.file.close()
        print("File closed automatically")

f = FileHandler("demo.txt")
f.write_data()

del f
```

File opened
File closed automatically

Double-click (or enter) to edit

Double-click (or enter) to edit

Double-click (or enter) to edit

What is Inheritance?

📁 Definition (Simple):

Inheritance is a mechanism where

📁 one class (child) can use properties of another class (parent)

Real-Life Example

📁 Father → Child

- Child gets:
 - name
 - property
 - behavior

📁 Same in programming:

Parent Class → Child Class

Double-click (or enter) to edit

Double-click (or enter) to edit

[1]

🎤 Start coding or generate with AI.



Colab paid products - Cancel contracts here

 Variables  Terminal



✓ 8:13 AM  Python 3